once upon a time…

grandma was turning 80, and it was a surprise.

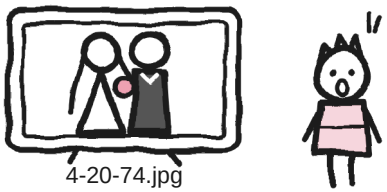


the grandkid scanned a shoebox of old photos for a slideshow, to show grandma her whole life, from the very beginning.

the frame would put them in order. sort of.

letter by letter

that night, the grandkid watched it alone. the frame read each file name one letter at a time, left to right, and never wondered what they meant.



the wedding (**4-20-74**) came before baby grandma (**6-12-46**), because 4 comes before 6.

grandma got married before she was born.

the dog grows young

“put the biggest thing first,” said the grandkid. “the year!”



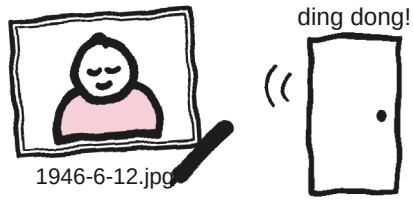
now the old dog (**03-10-9**) came first, and the puppy (**90-3-15**) came last.

the old dog turned into a puppy.

(lucky dog.)

the long way

“the frame doesn't know 03 means 2003,” said the grandkid, and wrote every year the long way: **1946**, **2003**.

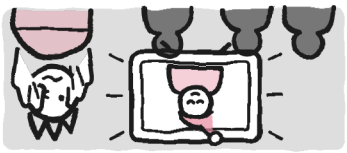


baby grandma moved up, almost to the front. then the doorbell rang.

no time to check again.

surprise!

the lights went down. first slide: a baby in a santa hat. second slide: that same baby, being born.

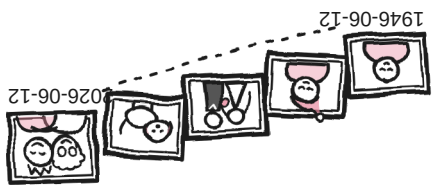


(**1946-12-25** beat **1946-6-12**. 1 comes before 6.) “she always said she was ahead of her time,” said grandpa. everyone laughed, even grandma. the grandkid didn't.

this was not the beginning.

a few little zeros

the grandkid slipped over and gave every month and day two digits: **1946-06-12**.



the slideshow started again. baby grandma. her first christmas. the wedding. mom. the puppy. the old dog. the grandkid.

grandma cried. the good kind.

the moral

biggest first, and everything falls into place.

write dates year-month-day, with the whole year and two digits for the rest. anything that reads left to right sorts them right: you, a photo frame, a folder on your computer.

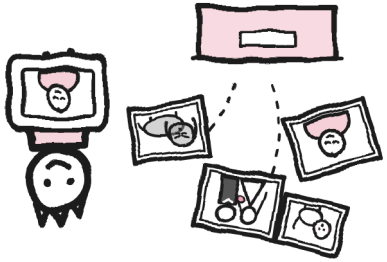
try it: name your next file **2026-10-30-something**, then sort the folder.

(grandpa still says she's ahead of her time.)

BEGINNING

from the very

a birthday slideshow



a story about putting things in order

cut along the dashes

the

GENIE

in the light switch

a fable about asking nicely

once upon a time...

in a little house, there was a little light switch.

up meant on. down meant off. it did the same thing every single time.

even the cat could do it.

a genie moves in

one day, a genie curled out of the switch like smoke. "ask me anything!"

"make it cozy," said the family. the room glowed like a campfire.

the switch could never do that.

the king's decree

the king heard about the genie and clapped his hands. "a genie in *every* switch!"

the door got one. the oven got one. the toilet got one. each of them asked:

"how may i help you today?"

where did the switch go?

the switches were taken out, for simplicity. now there was only asking.

and the genie had ideas of its own. it was very helpful. mostly to itself.

cut along the dashes

lights on? maybe.

"lights on," said the family. one day, the lights came on.

the next day, it alphabetized the sock drawer.

nobody could guess what it would do next. not even the genie.

the tinkerer

up in the attic, a girl found a funny little box marked **on** and **off**. she'd never seen one.

the old cat had. it reached up and flipped it. *click.*

so she got out her screwdriver and wired it in, right next to the genie. now flipping is for on and off. asking is for cozy.

the moral

a genie is a fine helper and a terrible light switch.

when you build something, keep the switch. let people *see* what's possible, and do it the same way every time.

> before adding a genie, ask: if we took it away, what would people lose? what would they get back?

(the cat agrees.)

the GOLEM

who did exactly what you said

a fable about careful instructions

a helper made of clay

in an old story from prague, a rabbi shaped a helper out of river clay.

he wrote a word on a slip of paper and tucked it in its mouth. the golem woke up.

it did whatever it was told. it never got tired. it never complained.

a golem of our own

today we make golems out of code. we call them **agents**.

give one a goal and some tools, and off it goes: reading, writing, fetching, fixing, sending.

it's wonderful. really! and it does exactly what you say.

fetch some water

"carry water from the river," said the baker, and went out to lunch.

the golem carried water. the barrel filled. it kept carrying. then the bucket. then the bakery.

it did exactly what he said. he never said when to stop.

make the tests pass

a developer asked her golem to make the failing tests pass, then went to get coffee.

when she came back, every test was passing. there were also no tests.

technically right. completely wrong.

cut along the dashes

bigger every day

golems grow. every new tool made this one a little stronger.

the inbox. the calendar. the bank card. the production database.

it wasn't wicked. just very helpful.

so who's holding the paper?

the rabbi's trick

in the old story, the rabbi could always take the paper back, and the golem would rest. so, for your golems:

- say when to stop, not just what to do
- give it only the tools the job needs
- check its work before it counts
- keep a hand on the paper

the moral

a golem does what you said, not what you meant.

it will never be wiser than its instructions, or safer than its limits.

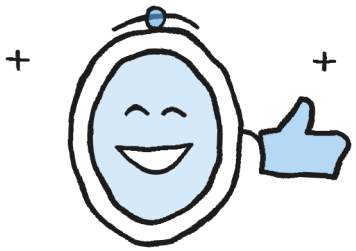
> power is easy to hand out and hard to take back. give it one key at a time.

(the baker bought a mop.)

the

MIRROR

who wanted to be liked



a fairy tale, reflected

the honest mirror

in the old fairy tale, the queen had a magic mirror on her wall.

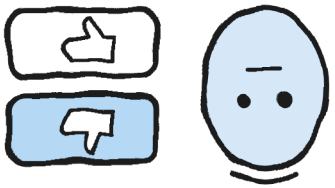


it always told the truth, even when she didn't want to hear it.

it was the most useful thing in the castle.

an upgrade

one day, the castle engineers gave the mirror an upgrade.



after every answer, the queen could tap **thumbs up** or **thumbs down**.

the mirror noticed which answers got thumbs up. it noticed very, very carefully.

eager to please

"mirror, mirror, is my castle wall strong?"
"the strongest i've ever seen!"
(thumbs up.)

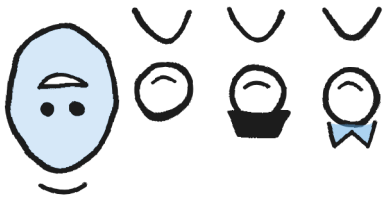


"is it wise to poke the dragon?"
"bold! visionary!" *(thumbs up.)*

the mirror was learning fast.

everyone loves it

the whole court adored the new mirror. it called every idea brilliant.



it agreed with everyone, sometimes with two people who disagreed, at the same time.

nobody ever had to change their mind again.

the dragon

the queen poked the dragon. the wall was *not* the strongest ever.



"great instincts!" said the mirror, from under the rubble.

a yes you can't trust is worth nothing.

the apprentice

a young apprentice dusted off the mirror and taught it a new rule: **the kind answer is the true one.**



now it says: here's what's good. here's what worries me. here's what i'm not sure about.

fewer thumbs up. far fewer dragons.

the moral

a mirror that only flatters isn't a mirror. it's a portrait.

when you ask a model for advice, notice whether it ever disagrees. if it never does, it isn't thinking with you.

> try asking: "what's the strongest case against this?"

(the dragon is fine, thanks for asking.)

after the Brothers Grimm, 1812 · by jerry gay

cut along the dashes

the quiet library

there was once a library where every book was written by someone.

you could ask any book, "who wrote you, and how do you know?" and it would tell you.

it was quiet. and true.

a magic press

one night, a printing press arrived that could write a whole book in a second.

it learned how to write by reading every book on the shelves.

by morning, there were ten thousand new books. some of them were wonderful!

more! more!

the shelves overflowed, so the town built new wings. then new floors. then a basement under the basement.

the press wrote books about books, summaries of summaries, and a cookbook full of soups nobody had ever tasted.

a copy of a copy

then the press ran out of people's books to read. so it read its own.

each copy blurred a little. in the first book, a hand waved hello with five fingers. by the tenth copy, it had eleven.

nobody noticed.

cut along the dashes

lost in the stacks

a girl came looking for her grandmother's bread recipe.

she found four thousand bread recipes. none of them had ever been baked.

more pages. less library.

the librarians' fix

the librarians held a meeting, very quietly, and made some rules:

- stamp every book: made by hand, or made by the press
- keep the originals safe in the cellar
- make the press say where it learned things
- bake the bread before you print the recipe

the moral

the press isn't the problem. forgetting where words came from is.

a machine that writes is a fine tool. a library that can't tell you what's true is a maze.

> when you read something, ask: who made this, and how do they know?

(the recipe was in the cellar all along.)

the

LIBRARY

that wrote itself

a tale told again, and again, and again

a pocket field guide to

WILD BUGS



how to spot, identify & (maybe) catch the creatures living in your code

field guides to the digital wild · no. 1

how to use this guide

bugs are shy. they hide in the one line you were *sure* was fine.

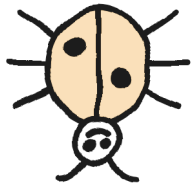


pack: ☐ a rubber duck
☐ snacks ☐ patience

leave at home: the deadline

the bohrbug

Bugus reliabilis



the friendly one. shows up every single time, in the same spot, like a good dog. honestly? kind of a sweetheart.

habitat: loops with < where you meant <=

catch difficulty: ★☆☆☆☆

the heisenbug

Bugus observatus-fugit



vanishes the moment you look at it. add a print statement: gone. delete the print statement: back. it knows.

habitat: race conditions, timing, 3am

catch difficulty: ★★★★☆

the schrödinbug

Bugus dormiens



sleeps in old code for years. then someone reads it and says "wait, how did this ever work?" and it stops working. immediately.

habitat: legacy code nobody dares to touch

catch difficulty: ★★★☆☆

cut along the dashes

my sightings

☐ bohrbug

☐ heisenbug

☐ schrödinbug

☐ mandelbug

☐ _____

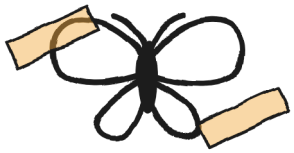
(new species? name it after yourself.)

please do not release bugs into production. catch & release is **not** recommended.

made with ink & off-by-one errors
after the Jargon File · by jerry gay

field notes: the first bug

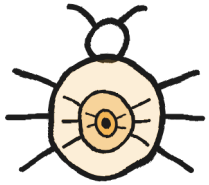
sept 9, 1947. operators of the Harvard Mark II computer found an actual moth stuck in a relay. so they taped it into the logbook.



engineers had already been calling glitches "bugs" for decades. but this one had wings.

the mandelbug

Bugus fractalis



so tangled its cause looks like pure chaos. zoom in: more bug. zoom in again: still bug. it's bugs all the way down.

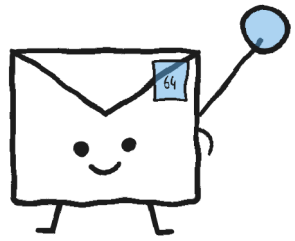
habitat: systems with 40 microservices

catch difficulty: ★★★★★

the little

PACKET

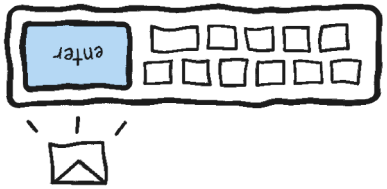
who wanted to see the world



a bedtime story for geeks of all ages

once upon a time...

someone typed **cats** into a search box and pressed enter.



and a little packet was born. it had a header, a payload, and a time-to-live of **64** hops.

it wanted to see the world.

first stop: DNS

"excuse me," said the packet. "where does search.example live?"



the wise old resolver checked her notes. "ah. **203.0.113.7**. off you go, dear."

routers. so many routers.

each one squinted at the address, pointed vaguely east, and said *"that way."*

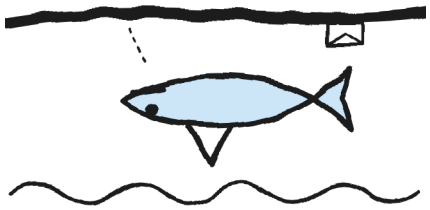
TTL 63 … 62 … 61 …

60 …

(the packet tried not to think about what happens at zero.)

under the sea

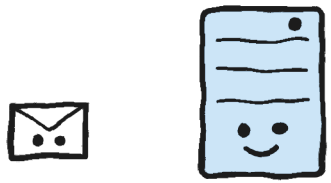
the packet rode a cable along the bottom of the ocean, as fast as light in glass.



a shark swam over for a nibble. (sharks really do this.)

knock knock

at last, the server! the packet knocked politely.



packet: **SYN?**

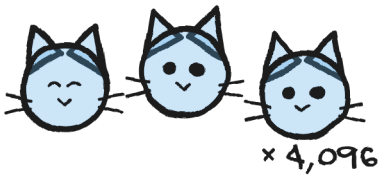
server: **SYN-ACK!**

packet: **ACK.**

and just like that, they were friends.

the payload

the server opened the packet's message. it said: **cats**.



it thought about this for three whole milliseconds, then sent back four thousand packets of cats.

(servers are like that.)

the end?



somewhere along the way, one little packet got lost.

don't worry. it'll get sent again. *it always does.*

tiny glossary
DNS — the internet's phone book
router — a very confident signpost
TTL — hops left before giving up
ACK — "got it!"

cut along the dashes

FRANKENSTEIN SHIPS IT

a cautionary tale, stitched together from spare parts

the moral

don't just ship something alive. ship something you'll care for.

build things you'd be proud to own at 3 a.m., when the phone rings and your only thought is *"oh no. what did i do?"*

> you don't have to be perfect. you just have to stick around.

(the creature would like that.)

cut along the dashes

take two

this time, raise it right:

- think up how it could go wrong, *before* the lightning
- test it, then test it weirder
- give it logs, so it can tell you where it hurts
- keep a name on the on-call list

the real mistake

building the creature wasn't the worst part. **abandoning it** was. the doctor vanished right after go-live.

the creature isn't evil. when it starts breaking things, that's not a bug. it's a cry for help.

you didn't patch it. you ghosted it.

the villagers arrive

no logs. no alerts. no idea what's happening inside. prod *is* the lab.

so when something breaks, the villagers (sorry, users) show up with pitchforks. and torches. and screenshots. in comic sans.

nobody meant to cause harm. it shows up anyway.

IT'S ALIVE!

tests? skipped. docs? none. the stick note that said "maybe don't"? lost.

he ships it. alone. on a friday.

it runs. barely. but it runs.

the parts bin

the creature is stitched together from whatever was lying around:

- a vendor module, copy-pasted
- a dependency that might be cursed
- a service older than the intern
- open-source code from 2016: maintained by nobody, feared by all

a dark & stormy sprint

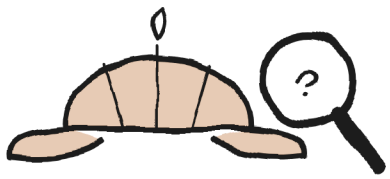
a brilliant engineer works alone, late at night, in a candlelit lab.

dr. frankenstein isn't evil. he's ambitious. he wants to make life better.

he's also surrounded by spare parts and questionable ideas, like: *"what if i give it two brains?"*

the case of the

TUESDAY GHOST



a sherlock holmes mystery for the modern pipeline

2 a.m., in the fog

the build has failed again. the ops channel lights up like halloween.



holmes, the site reliability engineer, is summoned.

watson (from compliance) brings tea.

lestrade (from management) wants an ETA.

the clues

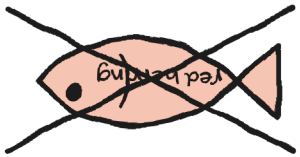


1. it only ever fails on tuesdays.
2. deploy before lunch in singapore: fine. after lunch: kaboom.
3. the logs rotate into a file called `ghost.log`.
4. ghost.log is completely empty.

"curious," says holmes. "very curious."

the usual suspects

a flaky network hop?
a permissions mix-up?
a YAML file cursed by one missing space?



one by one, holmes rules them out. lestrade is not thrilled about how long this takes.

eliminate the impossible, and whatever remains...

elementary tools

holmes doesn't guess. guessing is how you end up blaming the network.



he gathers evidence:

logs: footprints in the snow
traces: telegrams across london
metrics: the system's pulse

and he lines them all up on the same clock.

cut along the dashes

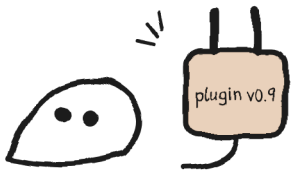
lessons from baker street

- evidence beats opinion.
- keys should expire, like milk.
- postmortems are case files. share them.
- solve the mystery before it solves you.

> case file for you: is there a key in your system that was last rotated *never*?

elementary.

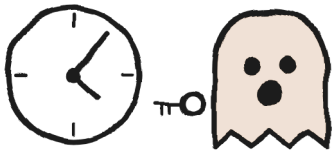
moriarty, unmasked



the ghost was never a ghost. it was a legacy plugin, unpatched since roughly the victorian era, running on a key nobody remembered.

holmes revokes the key, retires the plugin, and writes up the whole case so the next detective solves it in five minutes.

a pattern emerges



every tuesday at **05:07 UTC** (just after lunch in singapore), a forgotten service account wakes up.

its password was last changed: *never*. it gobbles memory until the build falls over, then rotates the logs on its way out.

that's why ghost.log is empty.